

RDSL: an Efficient Retrieval Algorithm for Group Editing CRDT

Yuran Ou (✉ hello@ouyuran.com)

Cloud Software Group

Jian Zhou

Cloud Software Group

Research Article

Keywords: Conflict-Free Replicated Data Types, Group editing, Skip list, Retrieval

Posted Date: September 6th, 2023

DOI: <https://doi.org/10.21203/rs.3.rs-3316287/v1>

License: © ⓘ This work is licensed under a Creative Commons Attribution 4.0 International License.

[Read Full License](#)

Additional Declarations: No competing interests reported.

RDSL: an Efficient Retrieval Algorithm for Group Editing CRDT

Yuran Ou^{1*} and Jian Zhou¹

^{1*}Cloud Software Group, No. 19 Suyuan Avenue, Nanjing, 210000, Jiangsu, China.

*Corresponding author(s). E-mail(s): hello@ouyuran.com;
Contributing authors: zhou136090@126.com;

Abstract

Conflict-Free Replicated Data Types (CRDT) is a popular approach for underpinning group editing in many applications, providing features such as peer-to-peer and offline editing. While conventional approaches such as Operation Transformation (OT) can be challenging to implement those features, CRDT still faces a critical issue - inefficient data retrieval due to storing data in a data structure with linear complexity. To address the issue, we propose a novel retrieval algorithm called Relative Distance Skip List (RDSL). This algorithm utilizes a probabilistic balancing identifier structure, which is based on relative distances between original CRDT nodes. The RDSL algorithm is both time- and space-efficient, providing a stable and efficient solution for data retrieval in CRDT.

Keywords: Conflict-Free Replicated Data Types, Group editing, Skip list, Retrieval

1 Introduction

Group editing [1] [2] [3] has become an integral part of many collaborative applications in today's interconnected world. It allows multiple users to edit a shared document simultaneously and independently, promoting a dynamic and interactive work environment. Group editing has been enabled through optimistic replication algorithms, which come in two main forms: Operational Transformation (OT) [4] [5] [6] and Conflict-Free Replicated Data Types (CRDT) [7].

OT has been a pivotal technique for handling concurrent operations in group editing systems, with its prominence demonstrated in widely-used collaborative editing

platforms such as Google Docs and Microsoft Office. These applications, structured in a client-server architecture, use OT to manage and resolve conflicts that arise from simultaneous edits, providing users with a seamless real-time editing experience.

However, the client-server model that OT relies on has significant limitations in peer-to-peer (P2P) environments. The inherent complexity of OT exhibits a quadratic increase when merging concurrent operations [8], making it less suited for P2P systems where a multitude of nodes may be operating and updating concurrently. Furthermore, the centralized nature of the client-server model can restrict scalability and robustness, which are critical factors in distributed and decentralized systems.

The limitations of OT in P2P environments underscore the advantages of CRDT. CRDT is a class of data types that allow replicas to be modified without coordinating with other replicas. When any two replicas have received the same set of updates, they deterministically reach the same state by following mathematically sound rules, thus ensuring state convergence. This characteristic positions CRDTs favorably in terms of performance during downstream operations, as they eliminate the need for complex transformations, leading to a more efficient and straightforward process.

Yet, CRDTs typically exhibit linear upstream complexity, which can compromise responsiveness when handling a large number of operations. Users often value prompt responses to their edits, and linear upstream complexity may hinder the real-time collaboration experience [9]. While auxiliary identifier data structures [10] have been proposed as a solution to enhance responsiveness, they can exacerbate memory overhead (although memory overhead is inherent to CRDTs). However, if the increase in memory overhead is insignificant, they hold promise as viable solutions.

In this paper, we introduce a novel identifier structure to CRDTs, aiming to boost responsiveness while minimizing memory overhead as much as possible. Moreover, this structure can be configured to strike a balance between time and space complexity, offering an efficient mechanism for managing concurrent operations in group editing systems.

This paper is organized as follows: Section 2 provides an overview of current CRDT algorithms and introduces a pre-existing identifier structure algorithm. In Section 3, we present the data structure and the corresponding algorithms underpinning our approach, as well as our strategy for handling block-wise scenarios. Section 4 offers an evaluation of our implementation, including a comparative analysis with existing alternatives. Possible enhancements to our approach are discussed in Section 5. Finally, in Section 6, we summarize the key findings and implications of our work.

2 Related works

2.1 Definitions

- **CRDT Document:** A CRDT document is a set of CRDT nodes. The organization of these nodes may vary based on the specific CRDT algorithm used.
- **CRDT Node:** A CRDT Node serves as the basic data structure. It includes one or more characters as well as additional metadata such as user ID and clock [11] [12].

- **Retrieval:** Retrieval refers to the act of locating a specific within a data structure, such as retrieving a CRDT node from a CRDT document based on an integer position p . This operation can be defined as $node = document.retrieval(p)$.
- **Operation:** An operation represents a modification to a CRDT document. This could be a delete, insert, or update operation. An insert operation involves adding a string s to a position p within the document, and can be defined as $document.insert(p, s)$. A delete operation involves removing n number of characters from the document, starting at the position p , and can be defined as $document.delete(p, n)$. An update operation involves replacing n number of characters from the position p with a given string, and can be defined as $document.update(p, n, s)$.
- **Upstream:** The process of applying local user operations and generating the corresponding operations to remote users. Any upstream operation should initially process a retrieval.
- **Downstream:** The process of applying operations received from remote users to a local CRDT document.
- **Tombstone:** A tombstone node refers to a deleted node that still exists in the CRDT document but has been marked as removed.
- **Block-wise:** A block-wise node is one that possesses the capability to contain more than a single character.

2.2 Conflict-free replicated data types

CRDTs have been extensively studied in the context of group editing. Several notable CRDT algorithms have been proposed, including WOOT, TreeDoc, Logoot, LSEQ, RGA, and YATA.

2.2.1 WOOT

WOOT [13] is considered one of the first CRDT algorithms for group editing. It utilizes a specific data structure called a W-character, which includes identifiers that describe the position of a character within the document. WOOT employs tombstones to handle deletions and has a linear time complexity for retrieval. WOOTO [14] and WOOTH [15] are variations that aim to optimize the performance of WOOT, but they still suffer from linear time complexity.

2.2.2 TreeDoc

TreeDoc [16] is a CRDT algorithm that utilizes a binary tree data structure for storing data. It demonstrates good retrieval performance under ideal conditions. However, it encounters difficulties when handling frequent insertions at the same position within the document. This can lead to an unbalanced tree structure and a subsequent decrease in overall performance. To address this issue, TreeDoc introduces a rebalancing algorithm. However, this rebalancing process requires synchronization between replicas, which impacts its status as a pure CRDT algorithm.

2.2.3 Logoot

Logoot [17] is a CRDT algorithm that assigns total order identifiers to each node. The original implementation stores these identifiers in an array, but more efficient data structures can be used to improve retrieval efficiency. However, Logoot faces challenges with identifier growth when multiple insertions occur at the same position.

2.2.4 LSEQ

LSEQ [18] can be considered an improvement to Logoot, employing a better identifier generation algorithm and utilizing an n-leaf tree data structure for storage. This makes it more efficient in retrieval. However, LSEQ still faces challenges with the memory overhead of identifiers compared to other CRDTs.

2.2.5 RGA

RGA [19] uses a linked list to store data, and each node is also stored in a hash map with its identifier as the key and a reference to itself as the value. This data structure makes downstream operations efficient, but the retrieval of upstream operations has a linear time complexity.

2.2.6 YATA

YATA [20] is another linked-list-based CRDT algorithm that features a special design merge algorithm. While the original paper suggests storing nodes in a balanced tree to improve retrieval efficiency, it does not provide detailed information on how to achieve that. Yjs, an open-source implementation of YATA by the same author, employs a linked-list data structure and leverages bookmarks to improve retrieval efficiency, although the time complexity remains linear.

2.3 Block-wise algorithm

Block-wise algorithms [21] [22] are a common enhancement to CRDTs. They allow multiple characters to be stored in a single CRDT node, increasing space efficiency. Operations in block-wise CRDTs may involve splitting nodes to accommodate new characters or mark existing characters as deleted.

2.4 Balanced tree identifier structure

Balanced tree identifier structures introduce auxiliary identifier structures [10] on top of the original CRDT data structure and can be adopted by most CRDT algorithms. They significantly improve retrieval efficiency. However, the creation of an identifier node for each CRDT node increases memory overhead, making it challenging to optimize memory usage. Additionally, the strict balancing principle of balanced tree structures can lead to performance degradation when frequent insertions occur at the same position. Deleting a node with many descendants can also adversely impact performance.

3 RDSL data structure and algorithms

Skip lists [23] have been proven to be an excellent choice for representing order lists due to their superior time and space efficiency, and ease of implementation. Many CRDT algorithms utilize a list for data storage and maintain a total order. However, these algorithms do not necessarily align their data storage order with this total order. This discrepancy presents a challenge to the direct adoption of skip lists within RDSL algorithms for enhancing data retrieval.

Relative distance skip list (RDSL) introduces a distance concept, represented as d , indicating the number of visible characters between two CRDT nodes (A and B). RDSL utilizes the distance-based concept to construct an identifier structure on top of a CRDT list. This identifier structure facilitates retrieval even in non-ordered lists. As the distance is relative, adding or updating necessitates modifying only a small portion of the identifier structure, thus enhancing efficiency.

Notably, d does not necessarily equal the number of nodes (m) between A and B due to variations such as 'tombstone' and 'block-wise' nodes. In cases where there are tombstone nodes between A and B , d is smaller than m . Conversely, if there are block-wise nodes between A and B , d is larger than m . With this distance information, we can employ an identifier data structure akin to a skip list for retrieval.

3.1 Data structure

The identifier data structure of RDSL comprises a number of RDSL nodes. Each node contains a reference to a related CRDT node, an array of distances, and an array of references to other RDSL nodes. The length of both arrays is determined by the level of the node. The level of an RDSL node, decided when a new CRDT node is created, is determined by a fraction p . The probability that a CRDT node has a level l RDSL node is p^l . If the level is 0, the RDSL node will not be created. Algorithm 1 elucidates the method for computing the random level.

Algorithm 1 Computing Random Level Procedure

```
1: function RANDOMLEVEL
2:   level  $\leftarrow$  0
3:   while random()  $< p$  and level  $<$  MAX_LEVEL do
4:     level  $\leftarrow$  level + 1
5:   end while
6:   return level
7: end function
```

For instance, In Figure 1, the document comprises 10 CRDT nodes ($C1$ - $C10$), with $C5$ serving as a block-wise node containing three visible characters, and $C6$ —marked with an 'X'—acting as a tombstone node with zero visible characters. All remaining nodes feature one visible character each. Additionally, there are four RDSL nodes ($R2$, $R4$, $R7$, and $R9$) of varying levels. Taking CRDT node $C7$ as an example, which has a corresponding level 2 RDSL node $R7$, the level 1 distance includes visible characters

from $C4$ to $C6$ —namely, $defg$ —resulting in a distance of 4. Level 2 extends to include $bcdefg$, thereby making the distance 6.

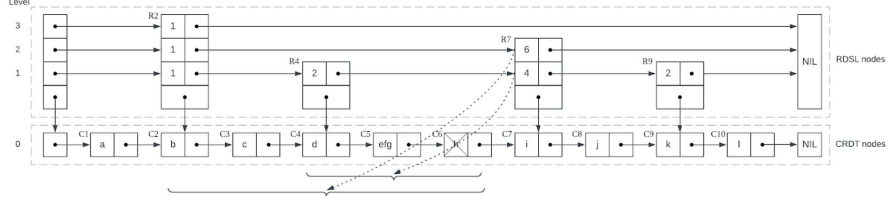


Fig. 1 Schematic diagram of algorithm 1

3.1.1 Replication and persistence of RDSL

RDSL data structures can vary among different replicas, even if the CRDT document is identical. In most cases, persistence of RDSL is not required. However, in instances where consistency or persistence is needed, RDSL can be easily reconstructed solely using the level of each CRDT node. This allows for easy transformation or storage of only RDSL level information, thus requiring minimal bandwidth and disk overhead.

3.2 Retrieval

The retrieval process requires a retrieval path, which serves the subsequent needs of RDSL addition and update algorithms. The path is defined as an array of the last node traversed during retrieval at each level, as well as the total distances traversed at each level. It can be defined as $path = document.retrieval(p)$. Retrieval commences from the RDSL head node, and a variable $posLeft$ is used to track the remaining distance to the retrieval end. $posLeft$ is initially equal to p .

Retrieval involves two types of moves: "moving right" and "moving down". If the distance to the next RDSL node at the current level is less than $posLeft$, we "move right". Otherwise, we "move down". "Moving right" signifies that we bypass a number of CRDT nodes, necessitating an update to $posLeft$. "Moving down" has two scenarios: 1) descending within an RDSL node, where no CRDT nodes are bypassed, and $posLeft$ does not require an update; 2) descending from an RDSL node to a CRDT node, where the current CRDT node is bypassed, and $posLeft$ must be updated. In both moves, we store the last node of each level and the total visible characters bypassed at each level in the retrieval path for future use. This process continues until $posLeft$ is equal to or less than 0.

For instance, Figure 2 illustrates an example of retrieval for position 7 in a document. The gray line depicts the traverse through the RDSL and CRDT nodes and the corresponding changes in $posLeft$. The green lines indicate the nodes that are ultimately stored in the retrieval path. The blue lines demonstrate the distances stored in the path.

Algorithm 2 Retrieval Procedure

```
1: function RETRIEVAL( $p$ )
2:    $\text{path} \leftarrow []$ 
3:    $\text{posLeft} \leftarrow p$ 
4:    $\text{currentNode} \leftarrow \text{RDSLHead}$ 
5:   while  $p > 0$  do
6:     if SHOULDGORIGHT( $\text{posLeft}$ ,  $\text{currentNode}$ ) then
7:        $\text{currentNode} \leftarrow \text{currentNode.references}[\text{currentNode.level}]$ 
8:       if  $\text{currentNode.level} > 0$  then  $\triangleright$  RDSL node
9:          $\text{posLeft} \leftarrow \text{posLeft} - \text{currentNode.distances}[\text{currentNode.level}]$ 
10:      else  $\triangleright$  CRDT node
11:         $\text{posLeft} \leftarrow \text{posLeft} - \text{currentNode.size}$ 
12:      end if
13:    else
14:       $\text{currentNode.level} \leftarrow \text{currentNode.level} - 1$ 
15:      if  $\text{currentNode.level} = 0$  then
16:         $\text{currentNode} \leftarrow \text{currentNode.crdtNodeReference}$ 
17:      end if
18:       $\text{posLeft} \leftarrow \text{posLeft} - \text{currentNode.size}$ 
19:    end if
20:     $\text{UPDATEPATH}(\text{path}, \text{currentNode})$ 
21:  end while
22:  return  $\text{path}$ 
23: end function
24:
25: function SHOULDGORIGHT( $\text{posLeft}$ ,  $\text{currentNode}$ )
26:    $\text{currentLevel} \leftarrow \text{currentNode.level}$ 
27:   if  $\text{currentNode.level} > 0$  then
28:     return  $\text{posLeft} > \text{currentNode.references}[\text{currentLevel}].\text{distances}[\text{currentLevel}]$ 
29:   else
30:     return  $\text{posLeft} > 0$ 
31:   end if
32: end function
33:
34: function UPDATEPATH( $\text{path}$ ,  $\text{currentNode}$ )
35:    $\text{currentLevel} \leftarrow \text{currentNode.level}$ 
36:    $\text{pathItem} \leftarrow \text{path}[\text{currentLevel}]$ 
37:   if  $\text{pathItem} = \text{NIL}$  then
38:      $\text{pathItem} \leftarrow \{\text{node} : \text{currentNode}, \text{totalDistance} : 0\}$ 
39:     if  $\text{currentLevel} = 0$  then
40:        $\text{pathItem.totalDistance} \leftarrow \text{currentNode.size}$ 
41:     end if
42:   else
43:      $\text{pathItem.node} \leftarrow \text{currentNode}$ 
44:     if  $\text{currentLevel} = 0$  then
45:        $\text{pathItem.totalDistance} \leftarrow \text{pathItem.totalDistance} + \text{currentNode.size}$ 
46:     else
47:        $\text{pathItem.totalDistance} \leftarrow \text{pathItem.totalDistance} +$   

 $\text{currentNode.distances}[\text{currentLevel}]$ 
48:     end if
49:   end if
50: end function
```

The retrieval process starts from the RDSL head node with $posLeft = 7$ and $currentLevel = 3$. Initially, the algorithm moves right to node $R2$, reducing $posLeft$ to 6. As node $R2$ has no reference at the current level, the algorithm moves down one level, keeping $posLeft$ unchanged at 6.

At the next stage, the algorithm encounters node $R4$ at level 2, which has a distance of 6. As $posLeft$ is not larger than this distance, the algorithm again moves down one level, where $posLeft$ remains 6.

Upon reaching node $R4$ at level 1, the algorithm can now move right, reducing $posLeft$ to 4. Unlike previous downward movements within the RDSL node that left $posLeft$ unchanged, this time the algorithm moves down from the RDSL node to the CRDT node $C4$, necessitating an update of $posLeft$ to 3. Now at $currentLevel = 0$, the algorithm continues to move right until $posLeft$ equals 0.

In block-wise CRDTs, the retrieval process may result in a negative $posLeft$ value. This value can be used to split the last CRDT node, a process that will be described in section 3.5.

In the retrieval process, we need to update the path in two scenarios: when moving down to a new level and when moving to the right.

When moving down to a new level, we create a new entry in the path array with the current node. If the level is greater than 0, the total distance for this new entry is initialized to 0 as the distance of the current RDSL node shouldn't be included. However, if the level is 0 (the CRDT node level), the initial total distance should be the size of the current CRDT node, as it should be included in the computation.

When moving right, we update the current node in the corresponding level's entry to the latest one and increment the total distance for that level accordingly.

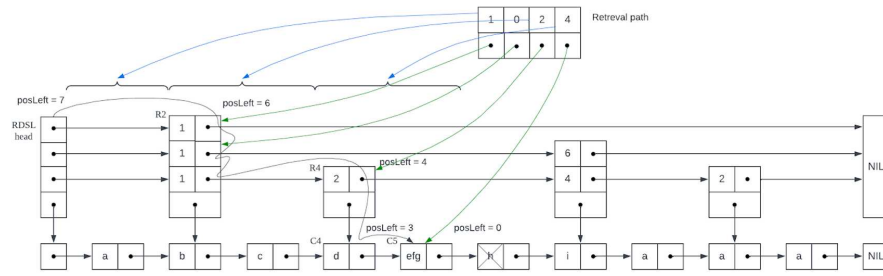


Fig. 2 Example of retrieval for position 7 in a document

3.3 Add RDSL node

When a CRDT node is added to the document, its level is first calculated (as per Algorithm 2). If the level is greater than 0, a corresponding RDSL node is created and added to the RDSL identifier structure. Regardless of whether a new RDSL node is created, existing RDSL nodes may need to have their distances updated.

Assume the total level of the RDSL identifier structure is N and the level of the current RDSL node is n ($0 < n \leq N$). For each level within n , we identify that the RDSL node in the retrieval path is the left node, and its reference node at that level is the right node. We should place the newly created RDSL node between the left and right nodes, calculate the distance for the newly created RDSL node, and update the distance in the right node accordingly. For RDSL nodes with a level greater than n , no references need to be updated. We only need to adjust the distances for the right node. This process is detailed in Algorithm 3.

For instance, as shown in Figure 3, consider inserting a new CRDT (C11) node with character 'X' at position 7 with a level 1 RDSL node ($R11$). We can get the $R4$ node as the left node from the retrieval path, and $R7$ as the right node in level 1. We need to update the references to place $R11$ between $R4$ and $R7$, and also adjust the level 1 and level 2 distances in $R7$ to fit the updated structure.

It's worth noting that multiple insert operations can follow a single retrieval. For example, we may execute n insert operations $I1 - In$, where $I1$ is inserted at position p and $I2$ is inserted after $I1$, and so on. In this case, a simple solution is to execute the operations in reverse order, from In to $I1$. This eliminates the need to update the retrieval path after the creation of any new RDSL node.

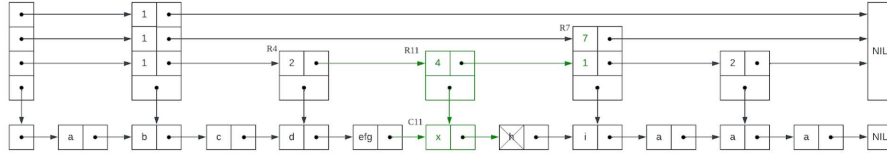


Fig. 3 Inserting a new CRDT node with character 'x' at postion 7 with a level 1 RDSL node

3.4 Update RDSL node

Certain CRDT algorithms may alter the number of visible characters in a CRDT node, for instance by marking a node as a tombstone or by modifying the characters within a node. For example, consider the scenario where we delete the character b at position 2 as shown in Figure 4, it results in the number of visible characters in $C2$ becoming 0. Consequently, we need to adjust the level 1 distance in $R4$ and the level 2 distance in $R7$. Figure 4 provides a schematic diagram illustrating the process outlined in Algorithm 4.

3.5 Block-wise support

The previous algorithms for retrieval, addition, and updating have all taken block-wise CRDT algorithms into consideration. One more scenario left to consider is the splitting of nodes. A block-wise CRDT node can be split into up to 3 parts. For instance, as depicted in Figure 5, using the consider a scenario where we delete the character f from the $C5$ node. The node will then be split into 3 distinct nodes:

Algorithm 3 Add RDSL Node Procedure

```
1: function ADDRDSLNode(crdtNode, path)
2:   level  $\leftarrow$  randomLevel()
3:   RDSL.totalLevel  $\leftarrow$  max(RDSL.totalLevel, level)
4:   if level > 0 then
5:     newRDSLNode  $\leftarrow$  new RDSLNode()
6:   end if
7:   for  $l \leftarrow 1; l \leq \text{RDSL.totalLevel}; l++$  do
8:     left  $\leftarrow$  path[l].node
9:     right  $\leftarrow$  leftNode.getRight(l)
10:    if  $l \leq \text{level}$  then
11:      left.references[l]  $\leftarrow$  newRDSLNode
12:      newRDSLNode.references[l]  $\leftarrow$  right
13:      newRDSLNode.updateDistance(l, calculateNewRDSLDistance(path, l))
14:      if right  $\neq$  NIL then
15:        right.updateDistance(l, crdtNode.size - newRDSLNode.distances[l])
16:      end if
17:    else
18:      if right  $\neq$  NIL then
19:        right.updateDistance(l, crdtNode.size)
20:      end if
21:    end if
22:  end for
23: end function
24:
25: function CALCULATENEWRDSLDistance(path, level)
26:   level  $\leftarrow$  —
27:   total  $\leftarrow$  0
28:   while level  $\geq$  0 do
29:     total  $\leftarrow$  total + path[level].totalDistance
30:     level  $\leftarrow$  —
31:   end while
32:   return total
33: end function
34:
35: function UPDATEDISTANCE(level, delta)
36:   this.distances[level]  $\leftarrow$  this.distances[level] + delta
37: end function
```

1. $C5 - left$: Contains the character e .
2. $C5 - middle$: Contains the deleted character f . This node is marked as a tombstone.
3. $C5 - right$: Contains the character g .

In our approach, we always let the left node ($C5 - left$ in this case) reuse the original CRDT and RDSL nodes, and then perform an update as described in the

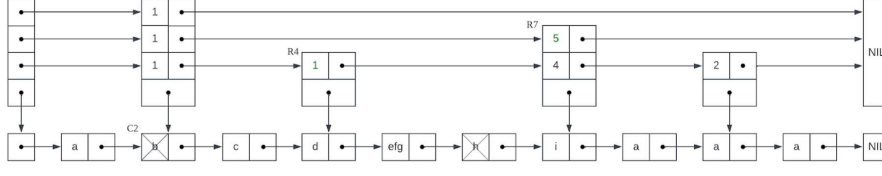


Fig. 4 Schematic diagram of Algorithm 4

Algorithm 4 Update RDSL Node Procedure

```

1: function UPDATERDSLNode(crdtNode, path)
2:   for  $l \leftarrow 1; l \leq \text{RDSL.totalLevel}; l++$  do
3:     left  $\leftarrow$  path[l].node
4:     right  $\leftarrow$  left.references[l]
5:     if right  $\neq$  NIL then
6:       right.updateDistance(l, delta)
7:     end if
8:   end for
9:   path[0].totalDistance  $\leftarrow$  path[0].totalDistance + delta
10: end function

```

section 3.3. Subsequently, we add the two new nodes from right to left. Thus, we add *C5-right* (C11 with a RDSL node R11) and *C5-middle* (C12) using the Algorithm 3.

4 Evaluation and experiments

In this section, we first evaluate the impact of p on our approach and then assess the time and memory complexity of the RDSL by comparing it with three classic CRDT algorithms: RGA, RGASplit (a block-wise RGA approach), and Logoot. We also compare these three CRDT algorithms with existing retrieval algorithms using a balanced tree approach: RGATree, RGASplitTree, and LogootTree. Our implemented versions are referred to as RGARDSL, RGASplitRDSL, and LogootRDSL.

We utilize a well-known Java benchmarking framework [8] [21] for our evaluation. This framework has been chosen as it has already been used by the original authors of many CRDT algorithms, as well as authors who have implemented their algorithm using the balanced tree approach.

4.1 Implementation

We implemented a shared RDSLNode class as an identifier structure and an RDSLWalker class to traverse RDSLNodes for retrieval. We extended existing algorithm classes and added an RDSL head node to the CRDT document class.

Each RDSLNode contains a reference array (storing the RDSLNodes adjacent to it), a 4-byte int array storing the relative distance to the previous RDSLNode, and a

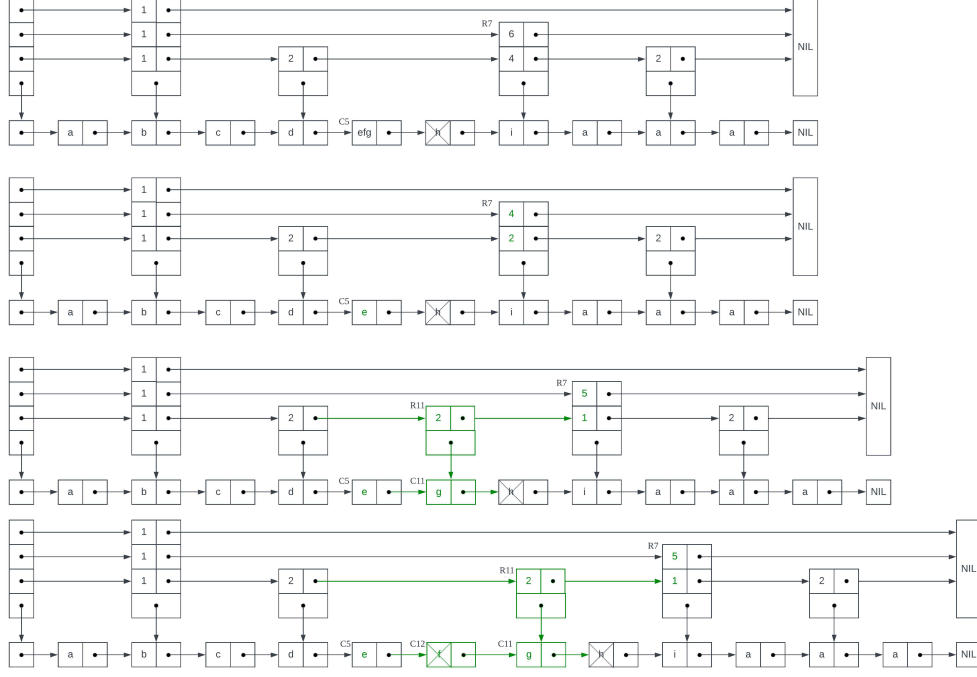


Fig. 5 Character deletion scenario: removing f from $C5$

reference to the original CRDT node. The lengths of these arrays depend on the level of the RDSLNode.

While the implementation could be further optimized by employing more efficient data structures for distance storage—considering that lower level distances are typically small and could be sizes of 1 or 2 bytes—the primary objective of this experiment is to illustrate how our approach can significantly enhance retrieval efficiency with minimal memory overhead.

All the implementations and benchmarking platform are publicly available at <https://github.com/ouyuran/replication-benchmark>.

4.2 Experiment design

The following experiments were performed on a MacBook Pro 16-inch 2019, equipped with a 2.6 GHz 6-Core Intel Core i7 processor, 32 GB 2667 MHz DDR4 memory, running the Ventura 13.0.1 OS. The JDK used was Oracle 17.0.5. As the algorithm is single-threaded, only one core was used, and 512 MB memory was allocated to the JVM.

Our approach primarily impacts upstream operations. For simplicity, we execute only local inserts. We generate random insert operations that follow a Gaussian normal distribution with a mean length of 10 and a standard deviation theta of 3.

We generated four different operation sets for all algorithms, with different operation numbers: 100, 1000, 10,000, and 100,000.

For each experiment, we measured the time spent executing all the insert operations. Time was measured using the Java *System.nanoTime()* method, while memory was measured using the *GraphLayout.totalSize()* method from *org.openjdk.jol*. Each algorithm was run 10 times to obtain an average value.

4.3 P impact

As anticipated, employing a smaller p results in smaller memory overhead. However, increasing the value of p to create a denser identifier structure does not always result in improved performance. It can lead to increased comparisons, more frequent retrieval path updates, and the creation of additional RDSL nodes. The optimal p value can vary based on the underlying CRDT algorithms. For example, in our experiment, RgaRDSL, RgaSplitRDSL, and LogootRDSL perform best when p is $\frac{1}{16}$, $\frac{1}{4}$, and $\frac{1}{8}$, respectively.

In practical applications, an appropriate p should be chosen based on the specific CRDT algorithm being used, and whether time or memory is of greater concern.

We conducted a comparative analysis, evaluating the P impact of our implemented CRDT algorithms, namely RGARDSL, RGASplitRDSL, and LogootRDSL, against existing retrieval algorithms utilizing a balanced tree approach: RGATree, RGASplit-Tree, and LogootTree. Figure 6 shows the comparison results. A p value of $\frac{1}{16}$ emerged as a balanced choice for most of the algorithms. Therefore, we opted to use $p = \frac{1}{16}$ for subsequent experiments. Remember, our approach may perform less efficiently when the number of operations is small due to additional computations, so we selected a sufficiently large operation set (10,000) for the test case analyzing P impact.

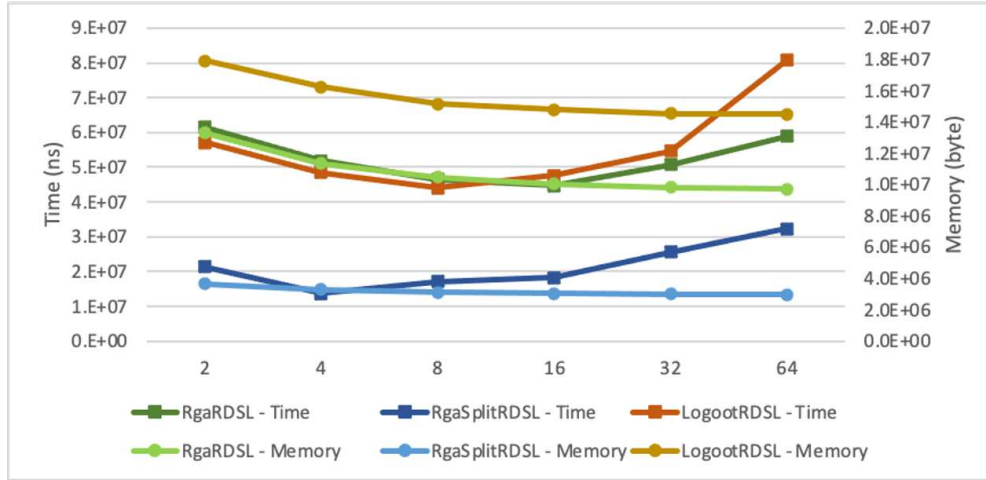


Fig. 6 Comparison results of P impact

4.4 Time complexity

We conducted a comparative analysis, evaluating the time complexity of our implemented CRDT algorithms, namely RGARDSL, RGASplitRDSL, and LogootRDSL, against original and existing retrieval algorithms utilizing a balanced tree approach: RGATree, RGASplitTree, and LogootTree. Figure 7 shows the comparison results.

Considering the time complexity, both the balanced tree approach and our method, which are $O(\log(N))$ algorithms, significantly reduce the total execution time as the number of operations increases. Our method is more complex when traversing the identifier structure but involves fewer identifier node creations. This results in our approach performing better than the Logoot but slightly worse than Rga and RgaSplit. However, the differences are marginal, and both identifier algorithms show significant improvements compared with the original CRDT algorithms.

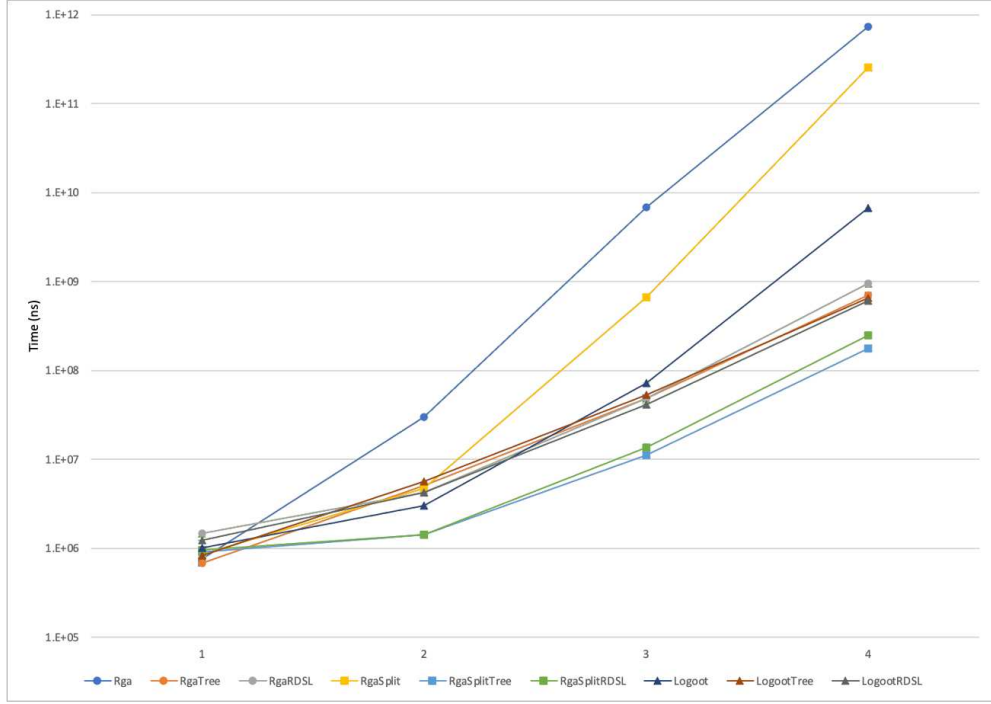


Fig. 7 Comparison based on time complexity

4.5 Space complexity

We conducted a comparative analysis, evaluating the space complexity of our implemented CRDT algorithms, namely RGARDSL, RGASplitRDSL, and LogootRDSL, against original and existing retrieval algorithms utilizing a balanced tree approach: RGATree, RGASplitTree, and LogootTree. Figure 8 shows the comparison results.

All algorithms have a memory complexity of $O(N)$. Hence, we focus only on the memory usage of the largest operation set, which consists of 100,000 random inserts.

First, it is noticeable that block-wise algorithms RgaSplit, RgaSplitTree, and RgaSplitRDSL consume less memory due to their capacity to consolidate multiple operations into a single CRDT node. Both the balanced tree approach and our method increase memory usage. However, due to the sparse identifier structure employed in our approach, it significantly mitigates memory overhead.

In comparison, the balanced tree approach uses an additional 20%, 18%, and 34% of memory for Rga, RgaSplit, and Logoot, respectively. On the other hand, our approach only requires an additional 4%, 3%, and 7% for these algorithms.

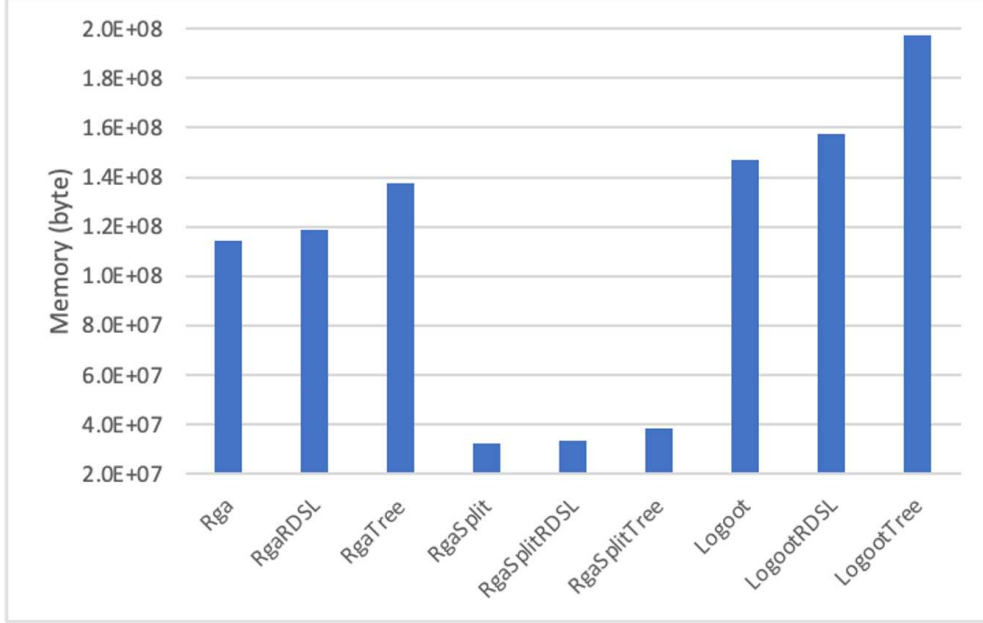


Fig. 8 Comparison based on space complexity

5 Further work

The RDSL data structure holds potential for further advancements in the following areas:

1. Adaptive structure: Currently, the RDSL identifier structure is probabilistic and balanced. However, in real-world scenarios, certain parts of a document may experience more intense activity, requiring more efficient retrieval. Aligning the identifier structure with those parts may enhance retrieval efficiency.
2. Lazy loading: Current RDSL algorithms assume that the entire document is loaded into memory, which can be challenging for extremely large documents. In traditional

editors, only the visible portion of the document is loaded to conserve resources. RDSL may enable lazy loading, where document sections are loaded on-demand, reducing unnecessary memory consumption.

3. Compression: Memory overhead is a critical concern for CRDT algorithms. While there are compression algorithms designed for persistent data storage, there is a lack of compression algorithms at run time. The RDSL identifier structure has the potential to serve as a "pattern breakpoint" for compressing CRDT metadata, reducing memory usage.

Continued exploration of these areas will contribute to the ongoing development and improvement of RDSL-based algorithms, further enhancing their efficiency and scalability.

6 Conclusion

In this paper, we present a novel approach, the Relative Distance Skip List (RDSL), to optimize the retrieval operation in CRDT algorithms. By integrating the RDSL Identifier structure into existing CRDTs, we significantly improve the retrieval efficiency, reducing the time complexity from $O(N)$ to $O(\log N)$. Our experimental evaluation demonstrates the effectiveness of the RDSL approach, showcasing its superior performance in terms of execution time and memory overhead compared to traditional CRDTs and balanced tree approach. The choice of the parameter 'p' plays a crucial role in balancing performance trade-offs. Overall, our work provides a promising solution to enhance the efficiency of CRDT algorithms, offering potential benefits for real-world applications.

References

- [1] Ellis, C.A., Gibbs, S.J.: Concurrency control in groupware systems. In: Proceedings of the 1989 ACM SIGMOD International Conference on Management of Data - SIGMOD '89, pp. 399–407. ACM Press. <https://doi.org/10.1145/67544.66963> . <http://portal.acm.org/citation.cfm?doid=67544.66963> Accessed 2023-05-19
- [2] Ellis, C.A., Gibbs, S.J., Rein, G.: Groupware: some issues and experiences **34**(1), 39–58 <https://doi.org/10.1145/99977.99987> . Accessed 2023-05-19
- [3] Greenberg, S., Marwood, D.: Real time groupware as a distributed system: concurrency control and its effect on the interface. In: Proceedings of the 1994 ACM Conference on Computer Supported Cooperative Work - CSCW '94, pp. 207–217. ACM Press. <https://doi.org/10.1145/192844.193011> . <http://portal.acm.org/citation.cfm?doid=192844.193011> Accessed 2023-05-19
- [4] Oster, G., Molli, P., Urso, P., Imine, A.: Tombstone transformation functions for ensuring consistency in collaborative editing systems. In: 2006 International Conference on Collaborative Computing: Networking, Applications and Work-sharing, pp. 1–10. IEEE. <https://doi.org/10.1109/COLCOM.2006.361867> . <http://ieeexplore.ieee.org/document/4207539/> Accessed 2023-05-19

- [5] Ressel, M., Nitsche-Ruhland, D., Gunzenhäuser, R.: An integrating, transformation-oriented approach to concurrency control and undo in group editors. In: Proceedings of the 1996 ACM Conference on Computer Supported Cooperative Work - CSCW '96, pp. 288–297. ACM Press. <https://doi.org/10.1145/240080.240305> . <http://portal.acm.org/citation.cfm?doid=240080.240305> Accessed 2023-05-19
- [6] Sun, C., Jia, X., Zhang, Y., Yang, Y., Chen, D.: Achieving convergence, causality preservation, and intention preservation in real-time cooperative editing systems **5**(1), 63–108 <https://doi.org/10.1145/274444.274447> . Accessed 2023-05-19
- [7] Preguiça, N.: Conflict-free replicated data types: An overview <https://doi.org/10.48550/ARXIV.1806.10254> . Publisher: arXiv Version Number: 1. Accessed 2023-05-19
- [8] Ahmed-Nacer, M., Ignat, C.-L., Oster, G., Roh, H.-G., Urso, P.: Evaluating CRDTs for real-time document editing. In: Proceedings of the 11th ACM Symposium on Document Engineering, pp. 103–112. ACM. <https://doi.org/10.1145/2034691.2034717> . <https://dl.acm.org/doi/10.1145/2034691.2034717> Accessed 2023-05-19
- [9] Shneiderman, B.: Response time and display rate in human performance with computers **16**(3), 265–285 <https://doi.org/10.1145/2514.2517> . Accessed 2023-05-19
- [10] Briot, L., Urso, P., Shapiro, M.: High responsiveness for group editing CRDTs. In: Proceedings of the 19th International Conference on Supporting Group Work, pp. 51–60. ACM. <https://doi.org/10.1145/2957276.2957300> . <https://dl.acm.org/doi/10.1145/2957276.2957300> Accessed 2023-05-19
- [11] Lamport, L.: Time, clocks, and the ordering of events in a distributed system **21**(7), 558–565 <https://doi.org/10.1145/359545.359563> . Accessed 2023-05-19
- [12] Mattern, F.: Virtual Time and Global States of Distributed Systems
- [13] Oster, G., Urso, P., Molli, P., Imine, A.: Data consistency for p2p collaborative editing. In: Proceedings of the 2006 20th Anniversary Conference on Computer Supported Cooperative Work, pp. 259–268. ACM. <https://doi.org/10.1145/1180875.1180916> . <https://dl.acm.org/doi/10.1145/1180875.1180916> Accessed 2023-05-19
- [14] Weiss, S., Urso, P., Molli, P.: Wooki: A p2p wiki-based collaborative writing tool. In: Benatallah, B., Casati, F., Georgakopoulos, D., Bartolini, C., Sadiq, W., Godart, C. (eds.) Web Information Systems Engineering – WISE 2007 vol. 4831, pp. 503–512. Springer. https://doi.org/10.1007/978-3-540-76993-4_42 . Series Title: Lecture Notes in Computer Science. http://link.springer.com/10.1007/978-3-540-76993-4_42 Accessed 2023-05-19

- [15] Ahmed-Nacer, M., Ignat, C.-L., Oster, G., Roh, H.-G., Urso, P.: Evaluating CRDTs for real-time document editing. In: Proceedings of the 11th ACM Symposium on Document Engineering, pp. 103–112. ACM. <https://doi.org/10.1145/2034691.2034717> . <https://dl.acm.org/doi/10.1145/2034691.2034717> Accessed 2023-05-19
- [16] Preguica, N., Marques, J.M., Shapiro, M., Letia, M.: A commutative replicated data type for cooperative editing. In: 2009 29th IEEE International Conference on Distributed Computing Systems, pp. 395–403. IEEE. <https://doi.org/10.1109/ICDCS.2009.20> . <http://ieeexplore.ieee.org/document/5158449/> Accessed 2023-05-19
- [17] Weiss, S., Urso, P., Molli, P.: Logoot: A scalable optimistic replication algorithm for collaborative editing on p2p networks. In: 2009 29th IEEE International Conference on Distributed Computing Systems, pp. 404–412. IEEE. <https://doi.org/10.1109/ICDCS.2009.75> . <http://ieeexplore.ieee.org/document/5158450/> Accessed 2023-05-19
- [18] Nédelec, B., Molli, P., Mostefaoui, A., Desmontils, E.: LSEQ: an adaptive structure for sequences in distributed collaborative editing. In: Proceedings of the 2013 ACM Symposium on Document Engineering, pp. 37–46. ACM. <https://doi.org/10.1145/2494266.2494278> . <https://dl.acm.org/doi/10.1145/2494266.2494278> Accessed 2023-05-19
- [19] Roh, H.-G., Jeon, M., Kim, J.-S., Lee, J.: Replicated abstract data types: Building blocks for collaborative applications **71**(3), 354–368 <https://doi.org/10.1016/j.jpdc.2010.12.006> . Accessed 2023-05-19
- [20] Nicolaescu, P., Jahns, K., Derntl, M., Klamma, R.: Near real-time peer-to-peer shared editing on extensible data types. In: Proceedings of the 19th International Conference on Supporting Group Work, pp. 39–49. ACM. <https://doi.org/10.1145/2957276.2957310> . <https://dl.acm.org/doi/10.1145/2957276.2957310> Accessed 2023-05-19
- [21] Andre, L., Martin, S., Oster, G., Ignat, C.-L.: Supporting adaptable granularity of changes for massive-scale collaborative editing. In: Proceedings of the 9th IEEE International Conference on Collaborative Computing: Networking, Applications and Worksharing. ICST. <https://doi.org/10.4108/icst.collaboratecom.2013.254123> . <http://eudl.eu/doi/10.4108/icst.collaboratecom.2013.254123> Accessed 2023-05-19
- [22] Lv, X., He, F., Cai, W., Cheng, Y.: A string-wise CRDT algorithm for smart and large-scale collaborative editing systems **33**, 397–409 <https://doi.org/10.1016/j.aei.2016.10.005> . Accessed 2023-05-19
- [23] Pugh, W.: Skip lists: a probabilistic alternative to balanced trees **33**(6), 668–676 <https://doi.org/10.1145/78973.78977> . Accessed 2023-05-19